Guía del notebook `22_opt_momentum`: el optimizador con momentum

Puntos: experimentos, sin `TODO` obligatorios · **Tiempo:** 20–25 min · **Módulo:** optimización (clase 6) · **Archivo:** `notebooks/22_opt_momentum.ipynb`
Requisito: `21_opt_gradiente_2d`.

¿Qué vas a aprender?

- Qué es el **momentum** y por qué ayuda en valles estrechos.
- A comparar dos optimizadores sobre una función difícil, la de **Beale**.
- Que el momentum permite usar tasas de aprendizaje con las que el descenso simple **se descontrola**.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/22_opt_momentum.ipynb` → kernel **nni**.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

El problema

La función de Beale tiene un **valle largo, estrecho y curvo**. Con el descenso simple, si la tasa es grande el algoritmo **rebota** contra las paredes del valle y se puede salir disparado; si es pequeña, avanza muy lento.

Momentum = recordar los pasos anteriores

En lugar de seguir solo el gradiente de este momento, el optimizador lleva un **promedio** de los gradientes recientes:

$m \leftarrow \beta \cdot m + (1 - \beta) \cdot \nabla f$ y parámetros $\leftarrow$ parámetros $- \eta \cdot m$

- **β** (beta, entre 0 y 1) es cuánta "memoria" tiene. Con $\beta = 0.9$, el paso de ahora se parece mucho a los anteriores.
- Los rebotes de un lado a otro **se cancelan** en el promedio, y la dirección constante, a lo largo del valle, **se mantiene**.

Analogía: una bola rodando por un valle no cambia de dirección bruscamente con cada piedrita; sigue su camino.

Función de Beale: su mínimo está en (3, 0.5) y vale 0. El notebook dibuja el **logaritmo** de la función para que se vea el valle.

Recorrido por las celdas

1. **Paquetes y problema:** `nni.ReducedBealeProblem()`, que dibuja x entre 0 y 4.5 e y entre −2 y 2.
2. **Gráficas:** contorno (con el valle curvo) y superficie 3D.
3. **Clase `MomentumOptimizer`:** hereda del descenso por gradiente y solo cambia `next_params` para usar la fórmula de arriba.
   El comentario `TODO: initialize dinamically` señala que `self.momentum = np.zeros(2)` supone 2 parámetros. Una mejora opcional: crear el vector del tamaño de `params` la primera vez que se llama a `next_params`.
4. **Hiperparámetros:** inicio (2.5, 1.0), tasa 0.01, 500 iteraciones y β = 0.9.
5. **Optimizar.**
   ✅ **Comprobación** (valores originales): termina ≈ **(2.922, 0.480)** con f ≈ **$1.07 \cdot 10^{-3}$**.
6. **Camino en el contorno** y **animación**, que necesita ffmpeg; ver el aviso de abajo.

Experimentos (resultados reales, desde (2.5, 1.0), 500 iteraciones)

Para comparar, el notebook trae comentada la línea del descenso simple: quita el `#` de esa línea y pon `#` en la del momentum.

Tasa	Descenso simple	Momentum β = 0.5	Momentum β = 0.9
0.01	f ≈ $6.95 \cdot 10^{-4}$	f ≈ $7.37 \cdot 10^{-4}$	f ≈ $1.07 \cdot 10^{-3}$
0.03	**diverge** (`OverflowError`)	f ≈ $1.9 \cdot 10^{-6}$ ✓	**f ≈ $8.0 \cdot 10^{-7}$** ✓

Cómo leer la tabla:

- Con tasa pequeña (0.01) los tres llegan casi igual. El momentum no acelera aquí, porque esta versión **promedia**: no amplifica los pasos.
- Con tasa grande (0.03) el descenso simple rebota hasta "explotar", mientras que el momentum **suaviza los rebotes** y llega prácticamente al mínimo (3, 0.5).
- **Conclusión:** el momentum permite usar tasas más grandes con seguridad.

⚠️ **Crea un optimizador nuevo cada vez que pruebes otros valores.** El objeto guarda su `momentum` de la ejecución anterior.

Para pensar

- ¿Qué pasa con β = 0.99? → Tiene demasiada memoria: tarda en frenar y en girar con la curva del valle (con tasa 0.01 queda en f ≈ 0.12).
- ¿Y con β = 0? → La fórmula se vuelve el descenso simple.

Errores frecuentes

Síntoma	Causa	Solución
`OverflowError: (34, 'Result too large')`	El descenso simple divergió con tasa grande	Es justamente lo que se quiere mostrar; usa momentum o baja la tasa

Síntoma	Causa	Solución
`RuntimeError: ... ffmpeg not available`	La animación necesita ffmpeg	Cambia a `.save('momentum_beta.gif', writer='pillow')`
Resultados raros al repetir	Reusaste el mismo optimizador	Crea uno nuevo en cada prueba
`AttributeError: ... 'GradientDescentOptimizer'`	`nni` viejo o notebook fuera del repo	`git pull` y abre la carpeta del repo